



Índices

// Práctica en clase



Ejercicio 1

1. Con la base de datos “**movies**”, se propone crear una tabla temporal llamada “TWD” y guardar en la misma los episodios de todas las temporadas de “The Walking Dead”.

```
CREATE TEMPORARY TABLE movies_db.TWD AS
SELECT e.*
FROM movies_db.episodes AS e
INNER JOIN movies_db.seasons AS t ON e.season_id = t.id
INNER JOIN movies_db.series AS s ON t.serie_id = s.id
WHERE s.title = 'The Walking Dead';
```

```
SELECT * FROM movies_db.TWD;
```

2. Realizar una consulta a la tabla temporal para ver los episodios de la primera temporada.

```
SELECT twd.*
FROM movies_db.TWD AS twd
INNER JOIN movies_db.seasons AS t ON t.id = twd.season_id
WHERE t.title LIKE "%Primer temporada%";
```



Ejercicio 2

1. En la base de datos “**movies**”, seleccionar una tabla donde crear un índice y luego chequear la creación del mismo.

```
CREATE INDEX movies_idx ON movies_db.movies (id);
```

2. Analizar por qué crearía un índice en la tabla indicada y con qué criterio se elige/n el/los campos.

Se crea un índice en la columna id de la tabla movies para acelerar las búsquedas y mejorar el rendimiento de consultas que usan WHERE, JOIN o ORDER BY con esa columna.

Se elige id porque es un campo único y se usa frecuentemente para identificar registros, lo que permite localizar datos de forma más rápida sin tener que recorrer toda la tabla.



// Práctica Grupal



Resolver las siguientes consignas

Tomando la base de datos **movies_db.sql**, se solicita:

1. Agregar una película a la tabla *movies*.

```
INSERT INTO `movies_db`.`movies` (`title`, `rating`, `awards`, `release_date`,  
`length`, `genre_id`) VALUES ('Pelicula agregada', '10', '3', '2012-05-04  
00:00:00', '120', '3');
```

2. Agregar un género a la tabla *genres*.

```
INSERT INTO `movies_db`.`genres` (`created_at`, `name`, `ranking`, `active`)  
VALUES ('2013-07-04 00:00:00', 'romance', '13', '1');
```

3. Asociar a la película del punto 1. genre el género creado en el punto 2.

```
UPDATE `movies_db`.`movies` SET `genre_id` = '13' WHERE (`id` = '22');
```

4. Modificar la tabla *actors* para que al menos un actor tenga como favorita la película agregada en el punto 1.

```
UPDATE `movies_db`.`actors` SET `favorite_movie_id` = '22' WHERE (`id` =  
'3');
```

5. Crear una **tabla temporal** copia de la tabla *movies*.

```
CREATE TEMPORARY TABLE movies_db.TablaTemporalMovies AS  
SELECT *  
FROM movies_db.movies;
```

6. Eliminar de esa tabla temporal todas las películas que hayan ganado menos de 5 awards.

```
DELETE FROM movies_db.TablaTemporalMovies  
WHERE awards < 5 AND id IS NOT NULL;
```

7. Obtener la lista de todos los géneros que tengan al menos una película.

```
SELECT DISTINCT g.*  
FROM movies_db.movies AS p  
INNER JOIN movies_db.genres AS g ON p.genre_id=g.id;
```

8. Obtener la lista de actores cuya película favorita haya ganado más de 3 awards.

```
SELECT a.*  
FROM movies_db.actors AS a  
INNER JOIN movies_db.movies AS p ON a.favorite_movie_id=p.id  
WHERE p.awards > 3;
```

9. Crear un índice sobre el nombre en la tabla *movies*.

```
CREATE INDEX idx_movies_name  
ON movies_db.movies (title);
```

10. Chequee que el índice fue creado correctamente.

```
SHOW INDEX FROM movies_db.movies;
```

11. En la base de datos **movies** ¿Existiría una mejora notable al crear índices?
Analizar y justificar la respuesta.

Sí, crear índices en la base de datos **movies** mejoraría el rendimiento, especialmente en consultas que buscan, ordenan o relacionan datos entre tablas. Los índices permiten encontrar información más rápido sin revisar toda la tabla. Sin embargo, pueden hacer que las inserciones y actualizaciones sean un poco más lentas porque el índice también debe actualizarse. En resumen, son muy útiles si se hacen muchas consultas, pero hay que usarlos con cuidado si hay muchos cambios en los datos.

12. ¿En qué otra tabla crearía un índice y por qué? Justificar la respuesta

Crearía un índice en las columnas que se usan con frecuencia en búsquedas, filtros o relaciones entre tablas, como claves foráneas o campos que aparecen en cláusulas **WHERE** o **JOIN**. Esto mejora la velocidad de las consultas al evitar que la base de datos revise toda la tabla para encontrar los datos.